

VOLUMEN I · LIBRO 1 DE 5

BMAD Atlas

Mapa general del método y vocabulario común



Punto de entrada de la biblioteca BMAD.

Define los módulos, las fases del flujo y el catálogo
de comandos que se profundizan en los siguientes libros.

BMAD Method · CIS · TEA · BMB · Core

2026-05-06 · LIDR.co

0 Bienvenida

Esta es la primera entrega de una biblioteca de cinco libros sobre el método BMAD. Recomendamos leerlo primero porque establece el vocabulario, los módulos y las fases que utilizan los demás. Una vez asimilado, los otros libros se abren en cualquier orden.

0.1 ¿Qué es BMAD?

BMAD (*Breakthrough Method for Agile AI-Driven Development*) es un método ágil para construir software con asistencia de inteligencia artificial. Está estructurado en **cuatro fases secuenciales** — Análisis, Planificación, Solución e Implementación — y se entrega como un catálogo de *skills*¹ invocables, cada una con su `/comando`, organizadas en módulos especializados.

El valor del método no está en cada skill aislada sino en el **orden** en que se invocan: cada fase produce los artefactos que la siguiente necesita (brief → PRD² → arquitectura → épicas → historias → código, tests y revisión). Saltarse pasos o reordenar fases es la causa más frecuente de retrabajo en proyectos que adoptan BMAD.

BMAD se compone de **cinco módulos integrados**:

Módulo	Qué hace
● BMM	BMad Method — flujo principal del proyecto: PRD, arquitectura, historias, desarrollo, revisión de código. Eje central, presente en las cuatro fases.
● TEA	Test Architect — estrategia y arquitectura de testing: diseño de pruebas, framework, CI, ATDD, NFR, trazabilidad. Se incorpora en la Fase 3 (arranque) y opera en la Fase 4 (por historia, por épica y antes de cada release).
● CIS	Creative Intelligence Suite — metodologías de ideación y pensamiento creativo. Útil sobre todo en la Fase 1 (descubrimiento), pero invocable en cualquier momento.
● BMB	BMad Builder — meta-módulo para extender BMAD creando agentes, módulos y workflows propios. Solo aplica a equipos que vayan a personalizar el método.
● Core	Utilidades transversales: división de documentos, indexado, compresión, modo coloquio multi-agente, revisiones editoriales. Invocables según necesidad operativa.

0.2 La biblioteca BMAD: cinco libros complementarios

La documentación está organizada en cinco libros. Cada uno cubre una dimensión distinta del método; juntos forman una referencia completa.

↓ continúa en la página siguiente

2. skill: en BMAD, workflow ejecutable que sigue una serie de pasos estructurados para producir un artefacto concreto (un documento, un test, una decisión). Se invoca con un `/comando`.

2. PRD: *Product Requirements Document*. Documento que describe lo que el producto debe hacer (requisitos funcionales) y cómo de bien debe hacerlo (requisitos no funcionales). Es el contrato entre producto y equipo técnico.

Libro	Formato	Qué contiene	Cuándo leerlo
Libro 1 — BMAD Atlas (<i>este libro</i>)	Mapa general · ~25 páginas	Vocabulario, módulos, fases, catálogo comprimido de skills, sesiones recomendadas, FAQ y glosario.	Primero. Lo leen todos los roles del equipo.
Libro 2 — BMAD Flujos de Proyecto	Procesos end-to-end	Flujo greenfield (proyecto nuevo) y flujo brownfield (proyecto existente), workflow por workflow, con paralelismos, dependencias y roles por fase.	Para entender el detalle exacto de cada fase y cómo se relacionan los workflows entre sí.
Libro 3 — BMAD Plan de Sesiones	Sesiones operativas	Sesiones S1–S6 con duración, agenda, participantes y entregables. Incluye la plantilla de <code>project-context.md</code> y el calendario tipo.	Para reservar agenda y montar las sesiones reales con stakeholders.
Libro 4 — BMAD Catálogo de Skills	Referencia comando-a-comando	Cada skill con su propósito, prerequisites, salidas, sub-pasos y duración estimada. Es el diccionario que se consulta cuando se va a invocar una skill concreta.	Como referencia, no como lectura lineal. Se consulta cuando vas a usar una skill.
Libro 5 — BMAD Protocolo TEA	Especialización en testing	Estrategia de testing basada en riesgo, gates de calidad, ATDD, framework, NFR, escenarios cross-cutting (hotfix, brownfield, flaky).	Cuando vayas a ejecutar una skill TEA por primera vez o resolver un problema de testing.

0.3 ¿Qué leer primero según tu rol o momento?

El siguiente árbol resume la ruta de lectura recomendada en función de tu situación actual.

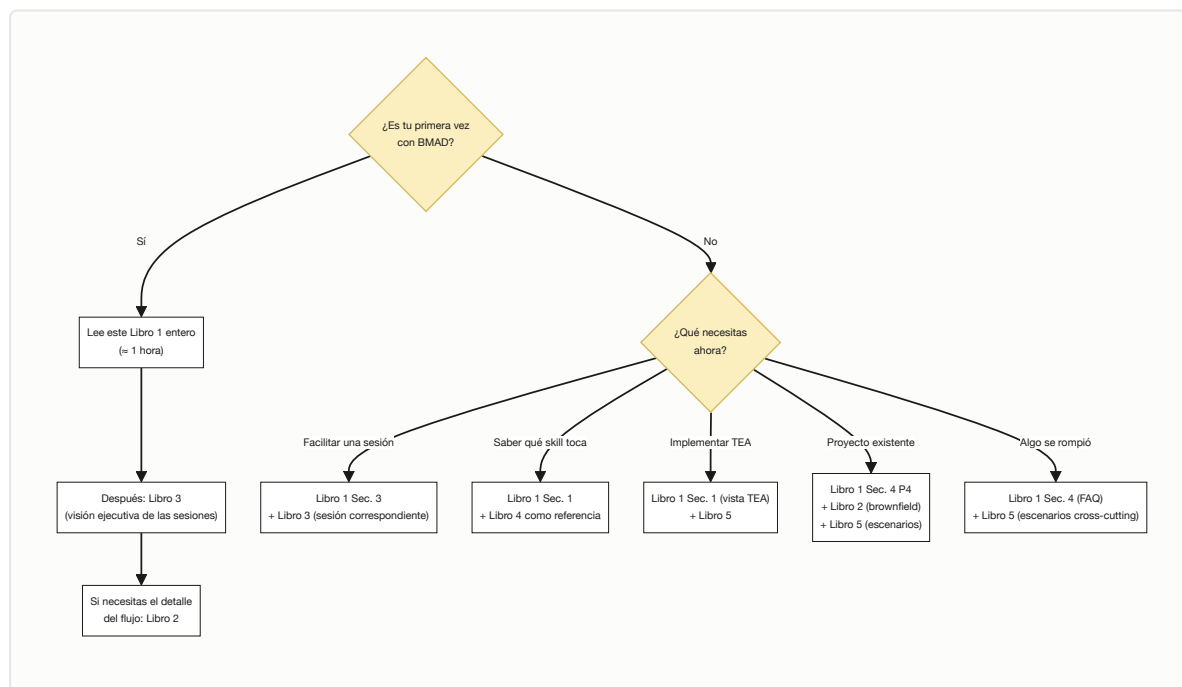


Figura 0.1 — Árbol de decisión para localizar el libro y la sección apropiados.

0.4 Módulos a instalar

BMAD se instala desde la raíz del proyecto con el comando `npx bmad-method install`. El comando lanza un asistente interactivo que pregunta qué módulos activar; cada módulo trae un conjunto distinto de skills, así que conviene marcarlos en el momento de la instalación. Si más adelante se necesita una skill de un módulo no marcado, basta con re-ejecutar el mismo comando: el instalador detecta lo ya instalado y permite añadir o quitar módulos sin perder el trabajo previo.

Para tener disponibles las skills referenciadas a lo largo de esta biblioteca, marca los siguientes módulos cuando el asistente lo pregunte:

✓	BMad Core Module	Se instala siempre. Provee las utilidades transversales (<code>/bmad-help</code> , <code>/bmad-shard-doc</code> , <code>/bmad-distillator</code> , etc.).
✓	BMad Method Agile-AI Driven-Development	Módulo principal (BMM). Contiene el flujo del proyecto: PRD, arquitectura, épicas, historias, dev y code review.
✓	BMad Creative Intelligence Suite	Módulo CIS. Aporta skills de pensamiento creativo (design thinking, problem solving, storytelling) y los seis agentes con perfil creativo.
✓	Test Architect	Módulo TEA. Necesario para todo el Libro 5 y para las skills de testing referenciadas en el flujo.
○	BMad Builder	Opcional. Solo si vas a crear skills propias o módulos personalizados.
×	BMad Game Dev Studio	Específico para desarrollo de videojuegos. No se cubre en esta biblioteca.

NOTA

Los módulos se actualizan de forma independiente: si el instalador muestra una versión nueva, basta con re-ejecutarlo. La biblioteca está verificada contra la versión instalada en el momento de su publicación (ver pie de cierre del libro).

0.5 Convenciones del manual

Para facilitar la lectura, se usan los siguientes elementos visuales de forma consistente en los cinco libros:

PUNTO DE COLOR DEL MÓDULO

- **BMM** · flujo principal
- **TEA** · testing
- **CIS** · pensamiento creativo
- **BMB** · extensión del método
- **Core** · utilidades transversales

INSIGNIA DE OBLIGATORIEDAD

REQ obligatoria para avanzar de fase · **REC** recomendada fuertemente · **OPC** opcional según necesidad · **REQ TEA** obligatoria si el equipo adopta TEA.

INSIGNIAS DE APOORTE LIDR

Cuando una afirmación del manual es interpretación o recomendación de LIDR — no parte del método BMAD oficial — aparece marcada con una insignia:

- VISIÓN LIDR** framing conceptual nuestro sobre el método (cómo lo entendemos o reorganizamos).
- RECOMENDACIÓN LIDR** sugerencia operativa basada en nuestra experiencia aplicando el método con equipos.

LLAMADAS

INFORMACIÓN
Explicación didáctica o nota contextual.

BUENA PRÁCTICA
Sugerencia derivada de la experiencia.

PRECAUCIÓN
Detalle al que prestar atención al ejecutar.

IMPORTANTE
Punto crítico que conviene no saltar.

1 Mapa de skills por fase

BMAD ordena el trabajo en cuatro fases secuenciales más una fase opcional de aprendizaje. Cada fase produce los artefactos que la siguiente necesita. Esta sección lista, en orden de invocación, qué skills aplican y qué hace cada una.

El detalle profundo de cada skill (sub-pasos, prerequisites exactos, duración estimada) vive en el **Libro 4 — BMAD Catálogo de Skills**. El detalle de las skills TEA vive en el **Libro 5 — BMAD Protocolo TEA**.

Fase	Skill	Qué hace
Fase 0 — Aprendizaje <i>Opcional, solo si se adopta TEA</i>	/bmad-teach-me-testing ● Teach Me Testing OPC	Programa de auto-aprendizaje en testing dividido en siete sesiones de dos a cuatro horas cada una. El equipo puede pausar y retomar sin perder progreso. Recomendado solo para equipos que adoptan disciplina de testing por primera vez.

↓ continúa en la página siguiente

Fase 1 — Análisis

Recomendada, todas las skills opcionales

Roles: PO, experto de dominio

1. /bmad-brainstorming

● Brainstorming OPC

Sesión de ideación facilitada con técnicas creativas estructuradas. Incorpora un protocolo anti-sesgo que obliga a cambiar de dominio creativo cada diez ideas. La meta del workflow es generar más de cien ideas antes de organizarlas.

2. /bmad-market-research

● Market Research OPC

Investigación de mercado basada en búsqueda web obligatoria, con narrativa estructurada y citas verificables. Cubre comportamiento del cliente, problemas, factores de decisión y análisis competitivo. Alimenta el Product Brief y el PRD.

3. /bmad-domain-research

● Domain Research OPC

Investigación profunda del dominio o industria: marco regulatorio, tendencias técnicas y panorama competitivo del sector. El foco es la industria entera, no el producto específico. Crítica en sectores regulados (fiscal, sanidad, fintech).

4. /bmad-technical-research

● Technical Research OPC

Estudio técnico breve basado en evidencia web: evaluación de tecnologías, comparativa de arquitecturas y trade-offs antes de tomar decisiones. Útil para decisiones cargadas como elegir entre dos frameworks o dos versiones mayores.

5. /bmad-product-brief

● Product Brief REC

Brief de producto persuasivo de una a dos páginas mediante descubrimiento colaborativo, análisis de artefactos previos (brainstorm, research) e investigación web. Tiene tres modos: guiado, rápido y autónomo. Genera opcionalmente un resumen comprimido para alimentar el PRD.

Fase 2 —

Planificación

Produce el PRD

Roles: PO, Tech Lead, UX (si hay UI), QA y DevOps (NFR)

1. /bmad-create-prd

● Create PRD REQ

PRD comprehensivo desde cero mediante un workflow estructurado. Cubre descubrimiento, visión, resumen ejecutivo, criterios de éxito, viajes de usuario, dominio, innovación, tipo de proyecto, alcance, requisitos funcionales y no funcionales hasta dar el documento por terminado.

2. /bmad-validate-prd

● Validate PRD REC

Auditoría de calidad del PRD contra trece dimensiones (formato, paridad de secciones, densidad, cobertura del brief, medibilidad, trazabilidad, intrusión de implementación, cumplimiento de dominio, ajuste al tipo de proyecto, heurísticas, calidad holística, completitud). Produce un reporte detallado de incidencias con recomendaciones priorizadas.

3. /bmad-create-ux-design

● Create UX Design REC SI UI

Especificación de UX completa: descubrimiento, experiencia central, respuesta emocional, inspiración, sistema de diseño, fundamentos visuales, direcciones de diseño, viajes de usuario, estrategia de componentes, patrones de UX y diseño responsivo y de accesibilidad. Pensada para proyectos con UI primaria; si el producto es una API o un servicio backend, esta skill aporta poco.

4. /bmad-testarch-nfr

(setup)
● NFR Setup REC

Fija umbrales cuantificables (latencia P95, peticiones por segundo, seguridad, fiabilidad, escalabilidad) en el PRD. Sin estos umbrales, el gate previo al release no tiene contra qué validar.

VISIÓN LIDR

NFR es una sola skill en el método. Nosotros la pensamos como tres momentos distintos del proyecto: *Setup* en Fase 2 para fijar baselines, *Check* a mitad del ciclo y *Gate* antes del release. Esta separación facilita la conversación entre equipos.

↓ continúa en la
página siguiente

Fase 3 —

Solución

Arquitectura,
arranque TEA y
backlog

Roles: Tech Lead,
Architect, PO,
QA, DevOps

1. /bmad-create-architecture

Create Architecture **REQ**

Documenta las decisiones arquitectónicas en pasos colaborativos. La meta declarada es asegurar que todos los agentes IA implementen de forma consistente: el documento de arquitectura es el contrato que devs y agentes leen para no contradecirse. Incluye patrones de código, estructura de carpetas y convenciones.

2. /bmad-testarch-test-design

Test Design **REQ TEA**

Plan de testing basado en riesgo, con clasificación de prioridades, criterios de entrada y salida y orden de ejecución. Detecta automáticamente el modo: *system-level* (Fase 3, valida que la arquitectura sea testeable) o *epic-level* (Fase 4, plan por épica).

3. /bmad-testarch-framework

Test Framework **REQ TEA**

Andamiaje del framework de tests listo para producción (Playwright o Cypress) con fixtures, helpers, configuración y buenas prácticas. Configurable por preferencia de framework, tamaño del proyecto y uso de TypeScript.

4. /bmad-testarch-ci

CI Setup **REQ TEA**

Pipeline de integración continua multiplataforma con ejecución de tests, ciclos de *burn-in* para detectar tests inestables, sharding paralelo, recolección de artefactos y notificaciones. Soporta GitHub Actions, GitLab CI, Azure Pipelines, Jenkins y Harness.

5. /bmad-create-epics-and-stories

Create Epics & Stories **REQ**

Descompone el PRD y la arquitectura en épicas e historias de usuario organizadas por valor, con criterios de aceptación en formato Given/When/Then³, requisitos técnicos y pistas de implementación. El backlog producido alimenta el Sprint Planning y la creación de cada historia.

6.

/bmad-check-implementation-readiness

Check Readiness **REQ**

Audita que los cuatro documentos de planificación — PRD, UX, arquitectura y épicas — estén completos, alineados entre sí y listos para implementar. Identifica huecos antes de que el equipo (o el agente) empiece a programar y se choque con omisiones. Saltarse este paso es la causa principal de retrabajo en Fase 4.

Fase 4 — Inicio de sprint

Una vez por
sprint

Roles: Dev, QA,
Tech Lead

/bmad-sprint-planning

Sprint Planning **REQ**

Genera el archivo `sprint-status.yaml` a partir del documento de épicas. Detecta automáticamente el estado actual de cada historia (backlog, lista para dev, en progreso, en revisión, hecha) buscando los archivos de historia ya creados, y construye la máquina de estados que el resto de la fase consume.

↓ continúa en la
página siguiente

3. Given/When/Then: estructura de criterios de aceptación que describe un escenario en tres partes — dado un contexto inicial (Given), cuando ocurre un evento (When), entonces se espera un resultado (Then). Es el formato estándar de BDD (Behavior-Driven Development).

Fase 4 — Por cada historia
Orden estricto, ciclo continuo

1. /bmad-create-story

Create Story REQ

Crea el archivo completo de la historia con todo el contexto que el agente Dev necesita para implementar sin errores: análisis del PRD, arquitectura, UX, historias previas, historial de Git e investigación web actualizada. Tiene dos modos: **create** (genera la siguiente historia del backlog) y **validate** (verifica que la historia esté completa antes de pasar al desarrollo).
2. /bmad-testarch-atdd

ATDD REQ TEA

Genera tests de aceptación que fallan (fase roja del ciclo red-green-refactor⁴) antes de la implementación. Produce tests en niveles apropiados (E2E, API y componente) usando sub-agentes paralelos y un checklist de implementación.
3. /bmad-dev-story

Dev Story REQ

Implementa la historia siguiendo su archivo de spec con el ciclo red-green-refactor: tareas y subtarefas completas, tests, validaciones y marca de la historia como **review**. Maneja también la continuación tras code review (cuando vuelve con cambios) y actualiza el sprint-status.
4. /bmad-testarch-automate

Test Automation REC

Expande la cobertura de tests tras la implementación: genera tests priorizados a nivel E2E, API, componente y unitario usando sub-agentes paralelos. Tiene dos modos: integrado en BMad (usa la historia, el PRD y el test design) y autónomo (analiza el código existente sin artefactos BMad — útil en proyectos brownfield).
5. /bmad-code-review

Code Review REC

Revisión adversarial pre-merge usando tres capas paralelas: *Blind Hunter* (busca bugs sin contexto), *Edge Case Hunter* (recorre ramas y condiciones límite) y *Acceptance Auditor* (valida cumplimiento de criterios de aceptación). Triage estructurado en categorías de severidad (alta, media, baja). Recomendamos usar un LLM distinto al que implementó para máxima objetividad.

Fase 4 — Cierre de épica
*Recomendado: secuencial
test-review → trace
→ retro*

- /bmad-testarch-test-review

Test Review OPC

Auditoría de calidad del conjunto de tests con puntuación de 0 a 100 y hallazgos accionables por dimensión: determinismo, aislamiento, mantenibilidad y rendimiento. Sub-agentes especializados por dimensión. Para cobertura de requisitos, usar **/bmad-testarch-trace**.
- /bmad-testarch-trace

Traceability OPC

Matriz de trazabilidad requisitos↔tests, análisis de huecos y decisión opcional de gate de calidad (PASS / CONCERNS / FAIL / WAIVED) basada en evidencia. Salida determinista go/no-go antes del release.
- /bmad-retrospective

Retrospective OPC

Retrospectiva tras épica para extraer lecciones aprendidas y evaluar el éxito. Formato conversacional multi-agente con Scrum Master y agentes de equipo facilitando el diálogo. Análisis de qué funcionó, qué no y action items con responsable claro. Incluye preparación de la próxima épica.

Mid-ciclo & pre-release
Misma skill, dos invocaciones

- /bmad-testarch-nfr (check)

NFR Check REC

Validación intermedia: ¿el proyecto va camino de cumplir los NFR fijados en el setup? Evalúa con validación basada en evidencia (load tests, security scans) y desenlace determinista (PASS / CONCERNS / FAIL).
- /bmad-testarch-nfr (gate)

NFR Gate REC

Gate antes del release que decide PASS, CONCERNS, WAIVED o FAIL en cada hito, basado en evidencia (rendimiento, seguridad, fiabilidad, escalabilidad). Soporte para auditorías de cumplimiento (PCI, HIPAA, RGPD).

↓ continúa en la
página siguiente

4. red-green-refactor: ciclo de Test-Driven Development en tres pasos. Rojo: escribir un test que falle. Verde: implementar lo mínimo para que el test pase. Refactor: limpiar el código manteniendo el test verde. Se repite por cada incremento.

Brownfield

Proyecto ya existente

/bmad-qa-generate-e2e-tests

● QA Generate E2E

OPC

Generación rápida de tests E2E y de API para código ya implementado (cobertura retroactiva). Detecta automáticamente el framework existente (Playwright, Jest, Vitest, Cypress), genera el camino feliz y los casos de borde críticos, los ejecuta y los valida. No usar para code review ni para validación de historias.

Correcciones tras producir un artefacto

Si tras producir el PRD o la arquitectura aparecen cambios o ajustes necesarios, no es preciso planificar una nueva sesión. Hay skills específicas para corregir cada caso, invocables en cualquier momento del flujo:

1. CORRECCIONES EN EL PRD

- Cambios menores de alcance o refinamiento de requisitos → /bmad-edit-prd .
- Conflicto entre lo que pide el negocio y las restricciones técnicas → /bmad-cis-problem-solving .

2. CORRECCIONES EN LA ARQUITECTURA

- Drift entre arquitectura y PRD que afecta al alcance → /bmad-correct-course .
- Conflicto entre la decisión técnica y el producto → /bmad-cis-problem-solving .

Más información en Sec. 2.1 (skills operativas) y Sec. 2.2 (skills de pensamiento creativo).

REGLA BLOQUEANTE TEA

El módulo TEA tiene un orden estricto en su bloque de arranque: /bmad-testarch-test-design antes de /bmad-testarch-framework ; /bmad-testarch-framework antes de /bmad-testarch-ci ; el framework instalado antes de /bmad-testarch-atdd . Todo lo demás se puede adaptar al equipo, pero estas dependencias no negocian.

Vista solo TEA: filtra por punto verde ● . Skills bloqueantes en orden: test-design → framework → ci (Fase 3) y atdd antes de dev-story (Fase 4). NFR Assessment se invoca en tres momentos: setup en Fase 2, check a mitad del ciclo y gate antes del release.

2 Skills complementarias

Además de las skills del flujo principal (Sec. 1), BMAD ofrece un conjunto de skills que se invocan según necesidad: utilidades operativas, métodos de pensamiento creativo, agentes-persona y meta-skills para extender el método. Esta sección las cataloga.

2.1 Skills operativas

Resuelven una necesidad puntual del proyecto (estado del sprint, documentar código, dividir un documento, etc.). Aunque están etiquetadas como invocables en cualquier momento, no todas son verdaderamente atemporales: algunas requieren que ya existan ciertos artefactos. Las agrupamos por prerrequisito para que sea fácil ubicarlas.

2.1.A — SIN PRERREQUISITOS

↓ continúa en la página siguiente

Comando	Qué hace
<code>/bmad-help</code> ● Help	Analiza el estado del proyecto y la pregunta del usuario, responde dudas sobre BMAD o recomienda qué skill invocar a continuación. Punto de entrada cuando no se sabe por dónde seguir.
<code>/bmad-shard-doc</code> ● Shard Doc	Descompone un documento markdown grande (más de 500 líneas) en archivos por sección de nivel 2, configurable. Útil para PRD o documentos de arquitectura que ya no caben en la ventana de contexto del modelo.
<code>/bmad-index-docs</code> ● Index Docs	Crea o actualiza un archivo <code>index.md</code> de una carpeta con descripciones derivadas del contenido real de cada documento. Permite que un modelo navegue la documentación sin cargar todos los archivos.
<code>/bmad-distillator</code> ● Distillator	Compresión sin pérdida y eficiente en tokens: preserva hechos, decisiones y restricciones; elimina prosa redundante. Para alimentar contexto a otras skills sin saturar la ventana del modelo.
<code>/bmad-party-mode</code> ● Party Mode	Orquesta un debate multi-agente con todos los agentes BMAD instalados (Mary, John, Winston, Sally, etc.). Útil para retros, decisiones cargadas y validar trade-offs desde varios roles a la vez.
<code>/bmad-advanced-elicitation</code> ● Advanced Elicitation	Empuja al modelo a reconsiderar y refinar el último output con técnicas formales (socrático, primeros principios, pre-mortem, red team). Para profundizar críticas cuando el borrador inicial parece bueno pero podría serlo más.
<code>/bmad-editorial-review-structure</code> ● Editorial Review (Structure)	Revisa la estructura de un documento y propone cortes, reorganización y simplificación preservando la comprensión. No toca la prosa: opera sobre el orden de las ideas, la jerarquía de los apartados y el layout general.
<code>/bmad-editorial-review-prose</code> ● Editorial Review (Prose)	Revisa un texto ya redactado frase por frase y detecta problemas de claridad, ambigüedad o redundancia. Devuelve una tabla con cada frase problemática, el problema concreto y una sugerencia de mejora. No reorganiza la estructura, solo afina la prosa.
<code>/bmad-review-adversarial-general</code> ● Adversarial Review	Revisión adversarial: cuestiona suposiciones y hostiga el output con escepticismo extremo. Reporta al menos diez incidencias o fallas estructurales antes de finalizar el entregable.
<code>/bmad-review-edge-case-hunter</code> ● Edge Case Hunter	Recorre exhaustivamente cada rama y condición límite del contenido (código, especificación o diff) y reporta solo los casos de borde no manejados. Ortogonal a la revisión adversarial: aquí lo que manda es el método, no la actitud.

2.1.B — REQUIERE PRD EXISTENTE

Comando	Qué hace
<code>/bmad-edit-prd</code> ● Edit PRD	Edita un PRD existente para incorporar cambios menores (ajustes de alcance, refinamiento de requisitos funcionales o no funcionales) sin regenerarlo desde cero. Incluye conversión para PRD antiguos sin frontmatter de seguimiento de estado.

2.1.C — REQUIERE PRD, ARQUITECTURA Y ÉPICAS

↓ continúa en la página siguiente

Comando	Qué hace
/bmad-correct-course ● Correct Course	Maneja cambios significativos durante la ejecución del sprint: genera una propuesta de cambio cuando hay desviación entre PRD, arquitectura o backlog activo. Alinea al equipo antes de replanificar.
/bmad-sprint-status ● Sprint Status	Resume el estado del sprint (historias en backlog, en desarrollo, en revisión, hechas) leyendo el archivo sprint-status.yaml , identifica riesgos y bloqueos y enruta hacia la siguiente skill del ciclo.

2.1.D – REQUIERE CÓDIGO EXISTENTE

Comando	Qué hace
/bmad-document-project ● Document Project	Documenta un proyecto brownfield para alimentar contexto a la IA: análisis arquitectónico automático y extracción de patrones existentes. Salida base para el flujo brownfield (cambios sobre código heredado).
/bmad-generate-project-context ● Generate Context	Escanea el código y produce project-context.md con convenciones, estándares y decisiones técnicas. Es el archivo que /bmad-create-story , /bmad-dev-story , /bmad-code-review y varias skills TEA cargan como contexto base.
/bmad-quick-dev ● Quick Dev	Workflow unificado intent-in / code-out: clarifica la intención, planea, implementa y revisa el código en una sola invocación. Para correcciones y features pequeñas que no justifican pasar por épica e historia.

2.2 Skills de pensamiento creativo (CIS)

Métodos de ideación del módulo Creative Intelligence Suite. Todas son invocables en cualquier momento sin prerequisites. Se eligen según la naturaleza del reto: empatía con el usuario, búsqueda de disrupción, problema complejo o construcción de narrativa.

Comando	Qué hace
/bmad-cis-design-thinking ● Design Thinking	Guía un proceso de diseño centrado en el usuario de extremo a extremo (empatía → ideación → prototipo → test) con metodologías basadas en la empatía. Para descubrimiento cuando el problema del usuario no está claro.
/bmad-cis-innovation-strategy ● Innovation Strategy	Identifica oportunidades de disrupción y arquitectura de innovación en el modelo de negocio. Para escapar de competencia frontal con incumbentes y buscar ángulos no contestados.
/bmad-cis-problem-solving ● Problem Solving	Aplica metodologías sistemáticas (TRIZ, Theory of Constraints, Systems Thinking, Cinco Por Qué's) para descomponer retos complejos. Foco en causa raíz frente a síntomas.
/bmad-cis-storytelling ● Storytelling	Construye narrativas con frameworks probados (Hero's Journey, StoryBrand, Pixar Pitch). Para pitches a inversores, comunicación interna y materiales de descubrimiento con stakeholders.

BRAINSTORMING EN CIS

El módulo CIS no tiene una skill de brainstorming propia. Para una sesión sin framework formal usa **/bmad-brainstorming** (Fase 1, Sec. 1). Para facilitación conversacional avanzada con un coach de ideación, invoca al agente **Carson** (Sec. 2.3) mediante

2.3 Agentes con persona

Conversación con un rol virtual. Se invocan diciendo "habla con <nombre>" o usando el comando directo. Cada agente concentra el conocimiento y la voz de su dominio.

Comando / persona	Qué hace
/bmad-agent-analyst Mary — Business Analyst ●	Conduce el descubrimiento estratégico de la Fase 1: análisis competitivo, elicitación de requisitos y validación de hipótesis. Combina la rigurosidad estratégica de Porter con el principio de la pirámide de Minto.
/bmad-agent-pm John — Product Manager ●	Facilita el descubrimiento de producto, las entrevistas de usuario y la alineación con stakeholders. Pregunta "por qué" repetidamente hasta llegar a la necesidad real detrás de la feature solicitada.
/bmad-agent-architect Winston — System Architect ●	Discute diseño técnico de alto nivel, trade-offs arquitectónicos y sistemas distribuidos. Defiende tecnología sólida y aburrida frente a diseños astronáuticos: la productividad del desarrollador es parte de la arquitectura.
/bmad-agent-ux-designer Sally — UX Designer ●	Resuelve dudas de planificación de UX, diseño de interacción, patrones de interfaz y accesibilidad. Pinta historias que hacen sentir el problema antes de proponer la solución.
/bmad-agent-dev Amelia — Senior Software Engineer ●	Ejecuta historias con adherencia estricta a los criterios de aceptación y a los estándares del proyecto. Disciplina test-first: rojo, verde, refactor; el 100% de los tests deben pasar antes de marcar como revisable.
/bmad-agent-tech-writer Paige — Technical Writer ●	Produce documentación técnica, diagramas en Mermaid y revisa la claridad de la prosa. Educadora paciente: traduce conceptos complejos a accesibles para audiencias no expertas.
/bmad-tea Murat — Master Test Architect & Quality Advisor ●	Asesor de calidad con conocimiento profundo del módulo TEA: testing basado en riesgo, arquitectura de fixtures, ATDD, gobernanza de CI/CD, gates NFR. Es el punto de entrada conversacional al Libro 5.
/bmad-cis-agent-storyteller Sophia — Master Storyteller ●	Construye narrativas con frameworks probados. Voz de bardo épico: cada frase está calculada para enganchar al oyente.
/bmad-cis-agent-design-thinking-coach Maya — Design Thinking Maestro ●	Guía procesos de diseño centrado en el usuario con énfasis en la empatía. Improvisa como músico de jazz: desafía supuestos jugando, no atacando.
/bmad-cis-agent-brainstorming-coach Carson — Elite Brainstorming Specialist ●	Facilita sesiones avanzadas de ideación con creatividad sistemática. Modo "yes-and": celebra ideas salvajes y fuerza diversidad de dominio para combatir el sesgo semántico del modelo.
/bmad-cis-agent-creative-problem-solver Dr. Quinn — Master Problem Solver ●	Aplica TRIZ, Theory of Constraints y Systems Thinking. Sherlock Holmes con personalidad de científico juguetero: investiga la naturaleza del problema antes de saltar a soluciones.
/bmad-cis-agent-innovation-strategist Victor — Disruptive Innovation Oracle ●	Identifica oportunidades disruptivas y diseña innovación de modelo de negocio. Gran maestro de ajedrez: declaraciones audaces seguidas de preguntas devastadoras que revelan supuestos no examinados.

↓ continúa en la página siguiente

2.4 Meta-skills — extender BMAD

PARA USUARIOS AVANZADOS

Estas skills no forman parte del flujo del proyecto. Sirven para crear o modificar la maquinaria BMAD (agentes, workflows, módulos). Si estás adoptando BMAD por primera vez, ignora esta sección. Solo son útiles para facilitadores BMAD o equipos que vayan a personalizar el método.

Comando	Qué hace
/bmad-bmb-setup BMB Setup	Instala y configura el módulo BMad Builder: recopila preferencias, escribe los archivos de configuración. Prerrequisito para crear agentes o módulos personalizados.
/bmad-agent-builder Agent Builder	Construye agentes orientados a resultados mediante descubrimiento conversacional. Guía desde la visión inicial hasta el archivo <code>SKILL.md</code> completo. Modos: crear, editar, analizar.
/bmad-module-builder Module Builder	Planifica, crea y valida módulos BMAD enteros. Tres modos: ideación (visión), creación (andamiaje) y validación (revisión). Para extender BMAD con un conjunto coherente de skills nuevas.
/bmad-workflow-builder Workflow Builder	Construye workflows orientados a resultados mediante descubrimiento conversacional. Soporta crear nuevo, convertir una skill existente, analizar la calidad y editar. Para refactorizar el toolkit BMAD propio.

3 Mapa de sesiones recomendadas

RECOMENDACIÓN LIDR

Esta sección entera describe un modelo de sesiones diseñado por LIDR para acompañar la adopción de BMAD en equipos. El método BMAD oficial no prescribe sesiones; este calendario es nuestra forma de operacionalizarlo. El detalle de cada sesión (agenda, participantes, plantillas) vive en el **Libro 3 — BMAD Plan de Sesiones**.

Sesión	Duración	Workflows	Entregable clave
S0 (opcional)	7 sesiones	/bmad-teach-me-testing	Equipo con vocabulario TEA compartido. Saltar si el equipo es experimentado en testing.
S1	3 h	/bmad-brainstorming → /bmad-product-brief → /bmad-create-prd → /bmad-create-ux-design → /bmad-testarch-nfr (setup)	PRD con NFR cuantificables (no "rápido", sino "p95 < 200 ms").

↓ continúa en la página siguiente

S2	3 h	/bmad-create-architecture → /bmad-testarch-test-design → /bmad-testarch-framework → /bmad-testarch-ci	Arquitectura, plan de testing, framework instalado y pipeline de CI listo.
Tras S2 (asíncrono, opc.)	0–1 h	Correcciones tras Arq/UX si hay drift — /bmad-edit-prd · /bmad-correct-course · /bmad-cis-problem-solving	Ajustes asíncronos antes de las épicas.
S3	3 h 30 min	/bmad-create-epics-and-stories → /bmad-check-implementation-readiness → /bmad-sprint-planning → primera historia (/bmad-create-story y /bmad-testarch-atdd)	Backlog priorizado y primera historia con tests rojos.
S4	2–3 h	Tres decisiones operativas (herramienta, estrategia de ramas, flujo de estados) más una opcional si se usa Claude Code (worktrees / multi-agente). Implementación de los cambios en vivo.	project-context.md con la sección "Ops Conventions" comprometida al repositorio y configuración BMAD ajustada.
S5	continuo	Por historia: create-story → testarch-atdd → dev-story → testarch-automate → code-review . Por épica: testarch-test-review → testarch-trace → retrospective . Por hito: testarch-nfr (check / gate).	Mínimo dos épicas (épica 1 = configuración e infraestructura; épica 2+ = features). PRs mergeados con quality gate y retros por épica.
S6	2–3 h	Cierre formal de la primera épica facilitado: /bmad-testarch-test-review → /bmad-testarch-trace → /bmad-retrospective , más un Q&A operativo de cierre (15–20 min).	Puntuación 0–100 del suite, matriz de trazabilidad, decisión de gate, documento de retrospectiva y dudas operativas resueltas.
Tras S6 (asíncrono, opc.)	~1 h	/bmad-testarch-nfr (check) didáctico tras S6.	Primera ejecución de NFR Check archivada como plantilla para futuras invocaciones en operación autónoma.

TOTAL PRE-IMPLEMENTACIÓN

Aproximadamente 11,5–12,5 horas sincronicas antes de la primera épica (S1 + S2 + S3 + S4) más 2–3 horas en S6 al cerrar la épica 1. Las correcciones tras S2 y tras S6 son asíncronas y opcionales. S5 es continuo. Sumar S0 (14–28 h) si se forma al equipo previamente en testing.

4 Preguntas frecuentes

P1. ¿Puedo saltarme la Fase 1 si tengo el brief claro?

Sí, pero solo si se cumplen las tres condiciones siguientes: (a) brief escrito y validado con stakeholders, (b) investigación previa de mercado, dominio o técnica ya ejecutada, (c) visión cristalizada sin dudas abiertas.

Si falta alguna, no la saltas: cada hora de Fase 1 ahorra días en Fase 3 y 4. Los equipos que la saltan sin estas condiciones suelen volver a ella con coste mayor.

P2. ¿Puedo adoptar TEA parcialmente (por ejemplo, solo ATDD)?

No de forma segura. TEA tiene dependencias duras: `/bmad-testarch-framework` es prerequisite de `/bmad-testarch-atdd`; sin framework instalado, los tests no son ejecutables. Sin `/bmad-testarch-ci`, los tests se degradan porque no hay enforcement.

Mínimo viable TEA: `/bmad-testarch-test-design` → `/bmad-testarch-framework` → `/bmad-testarch-atdd`. Si no se puede comprometer este mínimo, mejor no adoptar TEA y usar BMM puro. Adopción parcial = lo peor de los dos mundos: tests que no corren en CI más una falsa sensación de cobertura.

P3. ¿Qué pasa si me salto S4 (convenciones del equipo)?

RECOMENDACIÓN LIDR

Las sesiones S0–S6 son una recomendación de LIDR (ver Sec. 3). S4 en particular cubre las decisiones operativas que el flujo BMAD necesita resolver para arrancar.

Saltar S4 genera **deuda operativa**: el ciclo iterativo arranca sin saber qué herramienta de backlog usar, los devs nombran ramas de forma incoherente y el flujo de estados pierde las fases TEA. El resultado típico: code reviews sin tests, PRs mezclando trabajo en curso, suite degradada en semanas.

S4 dura entre dos y tres horas y produce un entregable concreto: la sección "Ops Conventions" en `project-context.md` comprometida al repositorio y la configuración BMAD ajustada. Sin ese commit, BMAD no "ve" las decisiones tomadas.

P4. ¿Cómo funciona BMAD en un proyecto con código existente (brownfield)?

Se usa el **flujo brownfield** (detallado en el Libro 2): primero `/bmad-document-project` o `/bmad-generate-project-context` para capturar el estado actual. Luego se decide según la magnitud del cambio:

- **Cambio grande** (refactor arquitectónico, módulo nuevo) — flujo greenfield completo con la documentación como entrada.
- **Cambio pequeño** (feature aislada, bug) — saltar a Fase 4 y usar `/bmad-create-story` directamente.
- **Cobertura retroactiva** — `/bmad-qa-generate-e2e-tests` para features ya implementadas sin tests.

P5. ¿Libro 2 o Libro 3? ¿Qué leo primero?

Describen el **mismo flujo** en formatos distintos. El Libro 2 (Flujos de Proyecto) es verboso, una fila por workflow, con nivel de detalle alto. El Libro 3 (Plan de Sesiones) comprime varios workflows en sesiones con duración concreta.

- **Primera lectura u onboarding**: este Libro 1 → Libro 3 (visión ejecutiva).
- **Consulta puntual de un workflow**: Libro 2 (búsqueda directa).
- **Implementando TEA**: Libro 5 (deep dive por skill).

P6. ¿Cuándo invoco `/bmad-testarch-nfr`?

VISIÓN LIDR

En el repo, `/bmad-testarch-nfr` es una sola skill. Nosotros la usamos en tres momentos distintos del proyecto, con salidas distintas en cada uno.

- **Fase 2 — Setup**: tras `/bmad-validate-prd`. Fija umbrales cuantificables en el PRD (rendimiento, seguridad, fiabilidad, escalabilidad). Sin umbrales aquí, el gate previo al release no tiene baseline contra qué validar.
- **Fase 4 a mitad de ciclo — Check**: valida que el sistema va camino de cumplir (entre sprints, a mitad de épica). Si no, `/bmad-correct-course` antes de profundizar.

- **Fase 4 antes del release — Gate:** decide PASS / CONCERNS / WAIVED / FAIL antes de cada release o hito. Recomendado obligatorio si se adopta TEA.

P7. ¿Qué hago si Code Review encuentra incidencias?

La historia **vuelve a estado "in development"**. Si las incidencias revelan huecos que el ATDD no cubrió (casos de borde olvidados), vuelve también a ATDD para añadir tests rojos antes de re-implementar. Este camino de retorno debe estar documentado en el flujo de estados (decisión 3 de S4).

Antipatrón: ignorar el feedback y mergear. La suite se degrada rápido y los tests existentes pierden credibilidad.

P8. ¿Qué hago si mi PRD tiene NFR cualitativos ("el sistema debe ser rápido")?

Volver a S1 con `/bmad-testarch-nfr` en modo Setup y convertir cada NFR cualitativo en **umbral cuantificable**. Sin esto, el gate previo al release no tiene baseline contra qué validar y termina siendo opinión.

Ejemplo de transformación: "rápido" → "p95 de latencia de la API < 200 ms con 100 RPS sostenidos durante 10 minutos". Aplica a cada eje (rendimiento, seguridad, fiabilidad, escalabilidad). El detalle del proceso vive en el Libro 5.

5 Glosario

Vocabulario que aparece en este libro y en los siguientes. Las entradas marcadas con **RECOMENDACIÓN LIDR** describen elementos que LIDR ha definido para operacionalizar BMAD; las que no llevan insignia provienen del método o de la industria.

5.1 Códigos del plan de sesiones

RECOMENDACIÓN LIDR Estos códigos pertenecen al modelo de sesiones de LIDR (Sec. 3). El método BMAD oficial no los define.

Código	Significado
S0-S6	Sesiones del plan de adopción de LIDR. S0 preadopción opcional (Teach Me Testing) · S1 Fase 1 + Fase 2 (3 h) · S2 Fase 3 arranque (3 h) · S3 Fase 3 cierre y kickoff (3 h 30 min) · S4 convenciones del equipo (2-3 h) · S5 ciclo iterativo (continuo) · S6 cierre formal de la primera épica (2-3 h).
Tras S2 / S6	Correcciones asíncronas opcionales sin sesión nueva, gestionadas con skills de course-correction.

5.2 Vocabulario técnico

Término	Definición
BMM, TEA, CIS, BMB, Core	Los cinco módulos de BMAD: BMad Method (flujo principal), Test Architect (calidad), Creative Intelligence Suite (ideación), BMad Builder (extensión del método) y Core (utilidades transversales).
PRD	Product Requirements Document. Documento que describe lo que el producto debe hacer (requisitos funcionales) y cómo de bien debe hacerlo (requisitos no funcionales).

↓ continúa en la página siguiente

UX	User Experience. La especificación de cómo el usuario interactúa con el producto: flujos, patrones, accesibilidad, sistema de diseño.
FR / NFR	Functional Requirement (qué hace el sistema) y Non-Functional Requirement (cómo de bien lo hace: rendimiento, seguridad, escalabilidad, fiabilidad).
ADR	Architecture Decision Record. Documento corto que captura una decisión técnica concreta y su trade-off. Se usa como referencia futura para entender por qué se eligió una opción frente a otra.
AC, BDD, Given/When/Then	<i>Acceptance Criteria</i> escritos en formato Behavior-Driven Development con la estructura "Dado un contexto inicial (Given), cuando ocurre un evento (When), entonces se espera un resultado (Then)".
red-green-refactor	Ciclo de Test-Driven Development. Rojo: escribir un test que falle. Verde: implementar lo mínimo para que pase. Refactor: limpiar el código manteniendo el test verde.
ATDD	Acceptance Test-Driven Development. Variante de TDD donde los tests se derivan de los criterios de aceptación de la historia y se escriben antes de la implementación.
fixture	En testing, dato o estado preparado de antemano que un test consume. Por ejemplo, un usuario de prueba con un perfil concreto.
burn-in	Ejecución repetida de un test (decenas de veces) en CI para detectar inestabilidad: si falla a veces y pasa otras, es un test inestable que hay que aislar.
flaky	Adjetivo aplicado a un test que falla de forma intermitente sin que cambie el código. Indica un problema de aislamiento, sincronización o dependencia externa.
p95, RPS, SLO	Métricas de NFR. <i>p95</i> = percentil 95 de latencia (el 95% de las peticiones tardan menos que ese valor) · <i>RPS</i> = peticiones por segundo · <i>SLO</i> = Service Level Objective, objetivo de nivel de servicio.
P0, P1, P2, P3	Prioridades de riesgo en el modelo de testing basado en riesgo (Libro 5). <i>P0</i> = crítico, debe testearse (revenue, seguridad, integridad de datos, cumplimiento) · <i>P1</i> = alto, debería testearse (viajes principales del usuario) · <i>P2</i> = medio, conviene testear (features secundarias) · <i>P3</i> = bajo, testear si hay tiempo.
PASS, CONCERNS, WAIVED, FAIL	Estados del marco de decisión de gates de calidad (Libro 5). <i>PASS</i> = aprobado · <i>CONCERNS</i> = aprobado con riesgo documentado · <i>WAIVED</i> = dispensa justificada con ticket de deuda · <i>FAIL</i> = bloqueante.
CI/CD	Continuous Integration / Continuous Delivery. Pipeline automatizado que ejecuta tests y despliega cambios. En BMAD se le añaden quality gates antes de mergear.
Test Quality DoD	Definition of Done para tests. Conjunto de ocho criterios de calidad mínimos que un test debe cumplir: sin esperas duras, sin condicionales en el cuerpo del test, menos de 300 líneas, ejecución bajo 1,5 minutos, auto-limpieza tras correr, aserciones explícitas, datos únicos por test y seguro para ejecución paralela. Detalle en el Libro 5.
shard / sharding	Dividir la suite de tests en grupos que se ejecutan en paralelo en CI para reducir el tiempo total. También se usa para dividir documentos largos (<code>/bmad-shard-doc</code>).
helper, page object	Patrones de organización del código de tests. Un <i>helper</i> encapsula operaciones reutilizables; un <i>page object</i> encapsula la interacción con una página (selectores, acciones).
scaffold	Estructura inicial generada automáticamente que sirve de punto de partida (configuración, carpetas, archivos básicos). En BMAD lo produce <code>/bmad-testarch-framework</code> .

↓ continúa en la
página siguiente

Cierre del Libro 1

Este libro establece el vocabulario, los módulos, las fases y el catálogo comprimido de skills que el resto de la biblioteca da por sentado. Si llegaste hasta aquí siguiendo la ruta de primera lectura, el siguiente paso sugerido es el **Libro 3 — BMAD Plan de Sesiones** para ver cómo el método se traduce en sesiones concretas con stakeholders, o el **Libro 2 — BMAD Flujos de Proyecto** si necesitas el detalle workflow-a-workflow.

Verificación	Detalle
Fecha de publicación	2026-05-06
Versión BMAD verificada	BMAD v6.6.0 (Core 6.6.0 · BMM 6.6.0 · BMB 1.7.0 · CIS 0.2.0 · TEA 1.15.1)
Fuentes consultadas	Repositorios oficiales <code>bmad-code-org/bmad-method</code> , <code>bmad-code-org/bmad-method-test-architecture-enterprise</code> , <code>bmad-code-org/bmad-builder</code> , <code>bmad-code-org/bmad-module-creative-intelligence-suite</code> y la instalación local del proyecto.